

Unit: II
Lecture: 6 (Part-II)
Garbage Collection Algorithms for equal sized blocks

Heap Memory:

Heap Memory has nothing to do with the heap data structure.

- *Heap data structure is a special tree-based structure in which the tree is a complete binary tree.*

Heaps are memory areas allocated to each program.

It is called heap because it is a pile of memory space available to programmers to allocate and de-allocate.



Memory allocated to heaps can be dynamically allocated, unlike memory allocated to stacks.

- Heap memory is global → it can be accessed and modified from wherever in the program it is allocated instead of being localized by the function in which it is allocated.
 - → Heap memory allocation is not safe.
- Heap memory is also known as dynamic memory.
- Dynamically allocated memory is referenced using pointers, which in turn leads to slight performance degradation over the use of local variables (on the stack).

Difference between Stack Allocation and Heap Allocation:

Stack Allocation:

This happens on contiguous blocks of memory. The size of memory to be allocated is known to the compiler and whenever a function is called, its variables get memory allocated on the stack. And whenever the function call is over, the memory for the variable is de-allocated.

A programmer does not have to worry about memory-allocation and de-allocation of stack variables. It is also known as temporary memory allocation

because as soon as the method finishes its execution all the data belonging to that method flushes out of the stack automatically.

Heap Allocation:

The memory is allocated during the execution of instructions written by programmers. Every time when we made an object it always creates in Heap-space and the referencing information to these objects is always stored in Stack-memory.

Garbage Collection Algorithms for Equal Sized blocks:

Mark-and-Sweep Algorithm:

Any garbage collection algorithm must perform 2 basic operations:

1. it should **be able to detect all the unreachable objects** and
2. it must **reclaim the heap space used by the garbage objects and make the space available again to the program.**

Both of the above operations are performed by mark and sweep algorithm in two phases:

- **Mark Phase**
- **Sweep Phase**

Mark Phase:

- When an object is created, its mark bit is set to 0 (False).
- **In the Mark phase, we set the marked bit for all the reachable objects to 1 (true).** To do this:
 - We perform a graph traversal i.e. a depth-first search approach is followed.
 - In this, we consider every object as a node and then all the nodes (objects) that are reachable from this node (object) are visited and it goes on till we have visited all the reachable nodes.
 - The root is a variable that refers to an object and is directly accessible by a local variable (assumption here is that we have one root only).

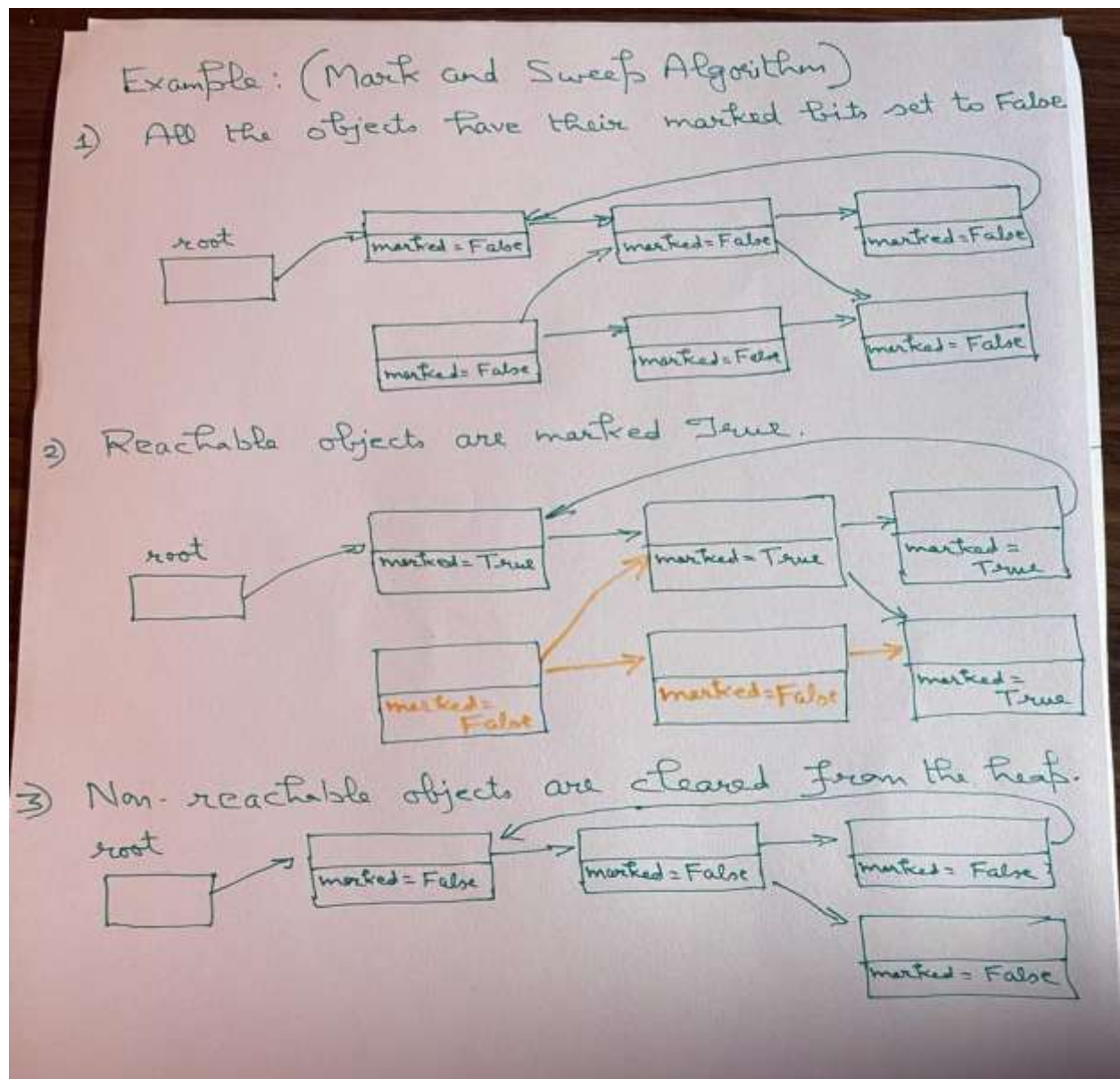
Sweep Phase:

- It sweeps the unreachable objects i.e. **it clears the heap memory for all the unreachable objects.**
- All those objects whose marked value is set to false are cleared from the heap memory, for all other objects (reachable objects) the marked bit is set to true.

- Now, mark value for all the reachable objects is set to false since we will run the algorithm (if required) and again we will go through the mark phase to mark all the reachable objects.

The mark and sweep algorithm is called a tracing garbage collector because it traces out the entire collection of objects that are directly or indirectly accessible by the program.

Example:



Advantages of Mark and Sweep Algorithm:

- It handles the case with cyclic references, even in the case of a cycle this algorithm never ends up in an infinite loop.
- There are not additional overheads incurred during the execution of the algorithm.

Disadvantages of Mark and Sweep Algorithm:

- The main disadvantage is that the normal program execution is suspended while the garbage collection algorithm runs.
- After the mark and sweep algorithm is run several times on a program, reachable objects end up being separated by many, small unused memory regions.